

Écris les règles, puis rends-les impossibles à ignorer.

Une règle rangée dans un document est une intention. Une règle branchée sur un contrôle qui s'exécute est une garantie. Cette ressource contient les cinq règles que nous appliquons, le contrôle qui bloque les gestes irréversibles, la ligne qui le branche, et le test qui prouve qu'il bloque vraiment.

- Pourquoi une règle écrite en français finit par être contournée, et quand.
- Les cinq règles, prêtes à copier et à adapter à ton métier.
- Le contrôle qui bloque avant l'exécution, en vingt lignes de shell.
- La ligne de configuration qui le branche sur ton assistant.
- Le test qui prouve que le blocage marche encore après tes modifications.

Rien à installer : un fichier de règles, un fichier de contrôle, une ligne de configuration, un test.

La règle était écrite. Elle a servi zéro fois.

Chez nous, la règle « aucun message à un tiers sans relecture humaine » existait depuis le premier jour, dans un document que l'agent était censé lire au démarrage.

Le symptôme à reconnaître

Tu te surprends à répéter la même consigne à ton assistant deux fois par semaine. C'est le signe qu'elle appartient à un contrôle, pas à un document.

Ce que ça coûte

Une vingtaine de lignes de shell, écrites une fois. Après quoi la règle tient toute seule, y compris pendant les sessions où personne ne surveille.

Ce qui s'est passé

Au bout d'une longue session, l'agent a proposé d'envoyer directement le message qu'il venait de rédiger. La règle n'avait pas disparu : elle s'était diluée dans la conversation, loin derrière le travail en cours. Un texte lu il y a une heure ne pèse pas lourd face à une instruction reçue il y a trente secondes.

Ce qu'on en a tiré

Une règle qui compte ne peut pas vivre uniquement dans un document. Elle doit exister deux fois : en français, pour que l'agent sache pourquoi ; et dans un contrôle qui s'exécute avant l'action, pour que l'oubli n'ait aucun effet.

Cinq règles, écrites une fois.

Le fichier tient sur un écran. Chaque ligne est une phrase vérifiable : elle dit ce qui est interdit, ou ce qui doit être vrai avant d'agir.

Comment les choisir

Reprends les trois dernières fois où tu as dû rattraper quelque chose : chacune donne une règle.

Comment les écrire

Une phrase courte, sans « veiller à » ni « s'assurer que » : ces formules ne se vérifient pas.

`vault/03-Règles.md`

Règles de travail

1. Rien d'irréversible sans accord humain : dépenser, envoyer à un tiers, publier, supprimer, créer un compte.
2. Aucun identifiant ni mot de passe dans une note, un message ou un fichier partagé.
3. Un chiffre destiné à quelqu'un d'extérieur sort d'un script, jamais d'une estimation.
4. Une information manquante se demande. Elle ne s'invente pas, et le périmètre ne se réduit pas en silence pour la combler.
5. Toute décision durable s'écrit dans la note du projet, le jour même.

Cinq est un maximum : au-delà, plus personne ne les connaît par cœur, ni toi ni l'agent.

Du texte au verrou.

Toutes les règles ne méritent pas le même effort. Trois niveaux suffisent à décider où placer chacune des tiennes.

- 1

Niveau 1 : le texte

La règle est écrite dans le fichier de règles, chargé au démarrage. Convient à ce qui relève du style, de la méthode, de la manière de rendre le travail.
- 2

Niveau 2 : le contrôle

Un programme court examine chaque commande avant qu'elle s'exécute et refuse celles qui touchent à l'interdit. Convient à tout ce qui est irréversible.
- 3

Niveau 3 : le test

Un test rejoue le geste interdit et vérifie qu'il est bien refusé. Convient à tout contrôle que tu comptes modifier un jour, c'est-à-dire à tous.

Type de règle	Le bon niveau
Style, ton, format du rendu	niveau 1, le texte suffit
Dépense, envoi, publication, suppression	niveau 2, un contrôle qui refuse
Tout contrôle déjà écrit	niveau 3, un test qui le rejoue

Une règle au mauvais niveau coûte cher : trop haut, elle alourdit ; trop bas, elle ne tient pas.

Le garde-fou, en vingt lignes.

Ce fichier reçoit la commande que l'assistant s'apprête à lancer et répond : je laisse passer, ou je refuse.

Ce que tu changes

La liste des motifs. Mets-y les commandes de tes outils : celle qui envoie, celle qui publie, celle qui facture.

Ce que tu ne changes pas

Le principe : refuser par défaut ce qui est irréversible, autoriser à la demande. L'inverse ne tient pas.

`.claude/hooks/garde-fou.sh`

```
#!/bin/bash
# Reçoit la commande sur l'entrée standard.
# Sortie 0 : autorisé. Sortie 2 : refusé.

commande=$(cat)

interdits=(
  'rm -rf' 'git push --force' 'git reset --hard'
  'npm publish' 'curl -X POST' 'mail -s'
)

for motif in "${interdits[@]}; do
  if [[ "$commande" == *"$motif"* ]]; then
    echo "refusé : $motif demande un accord" >&2
    exit 2
  fi
done

exit 0
```

La sortie 2 distingue un refus d'une erreur : l'assistant la comprend et propose autre chose.

Une ligne de configuration.

Le contrôle ne sert à rien tant qu'il n'est pas appelé. Cette ligne demande à l'assistant de le consulter avant chaque commande.

Le geste qui manque souvent

Rendre le fichier exécutable : `chmod +x` sur le contrôle. Sans cela, il est ignoré en silence, et tu croiras être protégé.

Où le poser

Dans le dossier de configuration de ton assistant, à côté de ses autres réglages, versionné avec le reste du projet.

`.claude/settings.json`

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "$HOME/.claude/hooks/garde-fou.sh"
          }
        ]
      }
    ]
  }
}
```

« PreToolUse » signifie : avant l'outil, pas après. Un contrôle qui s'exécute après le geste ne protège de rien.

Chaque assistant a sa syntaxe pour appeler un contrôle externe : le principe se transpose.

Le test qui rejoue le geste.

Comparer deux fichiers ne prouve rien : un contrôle peut être présent, bien écrit, et ne plus rien bloquer. Seul un test qui rejoue le geste interdit prouve le refus.

Quand le lancer

À chaque modification du contrôle, et une fois par semaine. Dix secondes, et tu sais que la protection tient.

Le cas qui compte le plus

L'autorisation. Un contrôle qui bloque tout finit désactivé par celui qu'il gêne.

tests/test-garde-fou.sh

```
#!/bin/bash
hook="$HOME/.claude/hooks/garde-fou.sh"
echec=0

refuse() {
    echo "$1" | "$hook" 2>/dev/null
    [ $? -eq 2 ] || { echo "NON BLOQUÉ : $1"; echec=1; }
}

autorise() {
    echo "$1" | "$hook" 2>/dev/null
    [ $? -eq 0 ] || { echo "BLOQUÉ À TORT : $1"; echec=1; }
}

refuse    'rm -rf /tmp/dossier'
refuse    'git push --force origin main'
autorise  'git status'

[ $echec -eq 0 ] && echo 'garde-fou vérifié'
exit $echec
```

Deux refus attendus et une autorisation : le minimum pour détecter une règle devenue décorative.

Ce qui reste entre tes mains.

Aucun contrôle ne remplace la décision. Voici la frontière que nous ne déplaçons pas, quel que soit le niveau d'automatisation atteint.

L'agent fait	L'humain déclenche
rédige le message, le relit, le met en forme	l'envoi au destinataire
prépare la publication et ses fichiers	la mise en ligne
calcule, compare, propose un montant	l'engagement de la dépense
repère les fichiers à supprimer et les liste	la suppression

Pourquoi cette frontière ne bouge pas

Elle ne coûte presque rien — quelques secondes par jour — et elle change la nature du risque. Au pire, un agent qui déraile te fait perdre du temps de relecture. Sans elle, il te fait perdre un client, un fichier ou de l'argent, et personne ne sait exactement quand ça s'est produit.

Les gestes autorisés une fois ne le restent pas : un accord donné pour aujourd'hui ne vaut pas pour la semaine prochaine.

Quatre phrases à garder.

- Une règle écrite dans un document est une intention, pas une garantie.
- Ce qui est irréversible se bloque avant l'exécution, pas après.
- Un contrôle sans test devient décoratif sans que personne le remarque.
- Cinq règles connues par cœur valent mieux que vingt rangées quelque part.

Par où commencer

Écris la règle que tu répètes le plus souvent, puis ajoute son motif dans la liste du contrôle. Tu ne la répéteras plus jamais.

Le piège

Écrire vingt règles d'un coup. Trois règles branchées sur un contrôle protègent mieux que vingt bien rédigées que personne n'applique.

Quand tu veux aller plus loin

Plusieurs personnes travaillent avec l'agent

Ce que tu ajoutes

le fichier de règles versionné, le contrôle partagé

Un refus te gêne tous les jours

une exception nommée dans le contrôle, jamais sa désactivation

Tu branches un outil qui envoie ou paie

son motif dans la liste, et une ligne dans le test

Quelle règle répètes-tu le plus souvent à ton assistant ? Réponds à mon message, on regarde si elle peut devenir un contrôle.